

Earnings Sentiment Platform

Architecture reference — current codebase, all phases complete

UPDATED JUNE 2026

Ingests public earnings-call transcripts from SEC EDGAR, scores CEO language confidence (0–100) using Claude, correlates those scores against post-call stock price returns, and surfaces everything in an interactive dashboard. **The central idea: how a CEO sounds on an earnings call often predicts short-term price movement.**

- | | |
|-----------------------------|--------------------------|
| 1 System overview | 7 Frontend pages |
| 2 Tech stack at a glance | 8 Key design decisions |
| 3 Data pipeline, end to end | 9 Local infrastructure |
| 4 Services in detail | 10 AWS deployment |
| 5 MongoDB data model | 11 Environment variables |
| 6 BFF API endpoints | |

1 System overview

Six layers take raw data from external sources to the user's browser. Each box below is either a running service or an external dependency.

EXTERNAL DATA SOURCES

SEC EDGAR

8-K filings · Atom RSS feed

FMP API

Transcript fallback · earnings calendar

Alpha Vantage

Transcript fallback · 8 quarters · 25 req/day · on-demand

yfinance

Post-call OHLCV price data

Claude API

Configurable model (scoring) · Sonnet (inspect)

Resend API

Portfolio email notifications



INGEST

Python Ingestor · port 8001

APScheduler cron · 3-pass strategy (EDGAR → FMP → Alpha Vantage) · 8-K RSS Atom poller · Flask HTTP trigger + rebackfill API · yfinance price fetch · price_refresh job · S3 archive · dedup tracker



STREAMING PIPELINE — KAFKA · 3 TOPICS · 3 PARTITIONS EACH

① Ingestor publishes two topics simultaneously

raw-transcripts

ticker · call_date · filing_id · raw_text · source

raw-prices

ticker · call_date · OHLCV dict keyed by date

② raw-transcripts flows through the Scoring Service and is published back as scored-transcripts

raw-transcripts → **Scoring Service** → scored-transcripts

checks scores collection · calls Claude · returns confidence_score · key_phrases · guidance_flag · trade_brief · qa_defensiveness

③ Correlation Service consumes both scored-transcripts and raw-prices

scored-transcripts + raw-prices → **Correlation Service**

yfinance · 1d/3d/7d returns · price_series · trend signal · sector denorm · raw_prices cache · 4-hour backfill daemon · Resend emails



STORE

MongoDB · 8 collections

companies · transcripts · scores · price_reactions · raw_prices · users · _ingested_filings · _sent_notifications



SERVE

Node.js BFF — Express REST API · port 3001

JWT auth · 12 route groups · /inspect SSE streaming · rate-limited Claude Sonnet deep analysis · FMP calendar proxy · on-demand ingestor trigger · sector analytics



CLIENT

Next.js 14 Frontend — App Router · port 3000

Dashboard · Company · Portfolio · Leaderboard · Sectors (inline detail) · Calendar · Login/Register ·
InspectModal (live SSE) · SearchOverlay

2 Tech stack at a glance

A one-line map of every layer before the deep dives. Each row is expanded later in the document.

LAYER	TECHNOLOGY	NOTES
Ingestion	Python · requests · APScheduler · Flask	EDGAR + FMP + Alpha Vantage (3-pass) + RSS Atom poller + HTTP trigger API (port 8001) + S3
Stream	Kafka — Confluent 7.5 local; Amazon MSK on AWS	3 topics · 3 partitions each · auto-create disabled
Scoring	Claude — model via <code>SCORING_MODEL</code>	Recommended <code>claude-haiku-4-5-20251001</code> · ephemeral prompt caching · idempotency check · 150k char cap
Deep analysis	Claude Sonnet — <code>claude-sonnet-4-6</code>	On-demand SSE streaming from BFF <code>/inspect</code>
Price data	yfinance	Post-call OHLCV · 12-day window · 4-hourly backfill · 2-hourly price_refresh publish
Earnings calendar	Financial Modeling Prep (FMP)	Also the transcript fallback source (Pass 2)
Transcript fallback	Alpha Vantage	Pass 3 — on-demand only (25 req/day free tier); last 8 quarters per ticker
Email	Resend API	Opt-in watchlist alerts; <code>NOTIFICATION_PROVIDER=resend</code>
Storage	MongoDB 7	8 collections · Mongoose ODM · Atlas in production

LAYER	TECHNOLOGY	NOTES
API	Node.js 20 · Express 4	JWT auth · 12 route groups · EDGAR-preferred dedup across all aggregations
Frontend	Next.js 14 (App Router) · React 18 · Tailwind · Recharts	Light theme · 8 pages · PortfolioContext · PredictabilityScatter · PostEarningsProfile
Local infra	Docker Compose	10 containers incl. kafka-setup one-shot + kafka-ui
Cloud	AWS — ECS Fargate · MSK · ECR · 2 × ALB · S3 · Secrets Manager · CloudWatch	Terraform IaC · GitHub Actions CI/CD (manual trigger)

3 Data pipeline — one transcript, end to end

Every step a single earnings filing goes through, from detection to appearing on the dashboard. This is the narrative spine of the whole system.

1 Detection

The SEC 8-K RSS Atom feed fires for a new filing, or the APScheduler per-ticker cron scan finds an unprocessed 8-K. The ingestor checks `_ingested_filings` — if this `filing_id` was already processed, it stops here. An on-demand `POST /trigger/<TICKER>` call can also start the flow immediately for a single ticker.



2 Fetch transcript + prices — 3-pass strategy

Pass 1 (EDGAR): downloads the 8-K exhibit and detects transcript text via signal phrases (operator calls, Q&A language). **Pass 2 (FMP):** always runs after EDGAR for structured transcript archives going back years; cross-source dedup via `_ingested_filings` and a `price_reactions` `call_date` check prevents double-publishing the same call. **Pass 3 (Alpha Vantage):** on-demand only (25 req/day free tier) — fetches the last 8 quarters for single-ticker trigger requests. Simultaneously, yfinance fetches OHLCV data (open, high, low, close, volume — the daily price bars) around the call date. Best-effort write to S3 (AWS only).



3 Publish to Kafka — two topics

Transcript text → `raw-transcripts` . OHLCV price dict → `raw-prices` . Both published simultaneously. `filing_id` is written to `_ingested_filings` to prevent any future re-publish. A separate `price_refresh` job (every 2h) re-publishes OHLCV for any filing that still has null return fields, so the correlation service can fill them in as each trading window closes.



4 Scoring service: raw-transcripts → Claude → scored-transcripts

Consumes `raw-transcripts` . Checks the `scores` collection first — skips records already scored with all fields present (records missing `trade_brief` or flagged `_mock` are re-scored). Calls Claude with a cached system prompt (`cache_control: ephemeral` cuts API cost). Model is set by `SCORING_MODEL` . Transcript is hard-capped at `MAX_TRANSCRIPT_CHARS` (default 150k). Returns five fields: `confidence_score` (0–100), three `key_phrases` , `guidance_flag` , `trade_brief` , `qa_defensiveness` . If Anthropic credits are depleted it commits and skips cleanly rather than blocking. Writes all five fields to `scores` , then publishes the enriched message to `scored-transcripts` .



5 Correlation service: scored-transcripts + raw-prices

The main thread consumes `scored-transcripts` , fetches 12+ days of post-call OHLCV from `yfinance`, and computes percentage returns at day 1, 3, and 7 from the baseline close (first trading day on/after the call). Builds an 8-point `price_series` array (day 0–7) for drift charts. Resolves sector via a 3-step cascade (`companies` collection → static S&P 100 map → `yfinance.info`). Computes a trend signal vs the ticker's previous call. Upserts to `price_reactions` keyed on `filing_id` (falls back to `(ticker, call_date)` for FMP records). Pending windows are left null for the backfill daemon. A separate thread caches `raw-prices` into `raw_prices` for the BFF chart endpoint.



6 Email notifications

After writing `price_reactions` , the notification system queries `users` for anyone watching this ticker with `notifications_enabled: true` , and sends an HTML email via Resend (only when `NOTIFICATION_PROVIDER=resend`). The `_sent_notifications` collection has a unique index on `(user_id, ticker, call_date)` — guaranteeing at-most-once delivery even across restarts.



7 BFF serves the call

`GET /feed` queries `price_reactions` within a 12-month window, deduplicated by `(ticker, call_date)` with EDGAR records preferred over FMP, sorted by `call_date DESC`. The new call appears as a feed card. If returns are still null (pending), a pending badge shows and the frontend polls every 60s.



8 Deep analysis on demand (streaming)

The user clicks a feed card → `InspectModal` opens → the frontend POSTs to `/inspect` → the BFF opens a Server-Sent Events (SSE) connection to Claude Sonnet and streams a five-section deep analysis to the browser in real time: *What Happened, Key Signals, What Drove the Price, Red Flags & Green Flags, Bottom Line*.

4 Services in detail

Python Ingestor

Three-pass transcript strategy — Pass 1: the EDGAR client downloads 8-K exhibits and detects transcripts via signal phrases; respects SEC fair-use limits (≤ 10 req/s, 0.12s delay). Pass 2: FMP structured transcript archives always run after EDGAR — cross-source dedup prevents re-publishing the same call. Pass 3: Alpha Vantage (25 req/day free tier) only during on-demand single-ticker triggers, to fill recent quarters.

RSS Atom poller — polls the SEC 8-K Atom feed every `RSS_POLL_INTERVAL_MINUTES` (default 10 min). Catches *any* public company that files, even those not in `TICKERS`. Triggers a background historical backfill per newly discovered ticker. This is the primary real-time coverage mechanism; the per-ticker scan is a reliability backstop.

Flask HTTP API (port 8001) — four endpoints. `POST /trigger/<TICKER>` starts an immediate on-demand scan (EDGAR 90-day + full FMP archive + Alpha Vantage 8 quarters + async historical backfill) — called by the BFF's ingest endpoint. `POST /rebackfill/<TICKER>` clears all processed flags for one ticker and re-ingests it. `POST /rebackfill-all` does the same for every tracked ticker. `GET /health` is the liveness probe.

Scheduled jobs (APScheduler) — `periodic_ingest`: every `SCHEDULE_INTERVAL_HOURS` (default 2h), runs Pass 1+2 for all tickers (static + user watchlists + recently active). `price_refresh`: every 2h, re-publishes OHLCV for any filing still missing returns.

Watchlist expansion — each scheduled run reads all users' watchlists from MongoDB and merges those tickers into the scan set, so newly bookmarked companies are covered automatically.

S3 archival — best-effort transcript backup to S3 when running on AWS. No-op locally.

Scoring Service

Claude scoring prompt — uses a cached system prompt (`cache_control: ephemeral`) so it's billed once per cache window. Returns a structured JSON object with five fields:

- `confidence_score` — integer 0–100. Five-band rubric: 90–100 exceptionally confident · 70–89 confident · 50–69 neutral · 30–49 cautious · 0–29 very uncertain.
- `key_phrases` — three verbatim/near-verbatim phrases (≤ 12 words each) that most influenced the score.
- `guidance_flag` — direction of forward guidance: `raised` · `maintained` · `lowered` · `withdrawn` · `null` .
- `trade_brief` — two plain-English sentences for a short-term trader: what the tone signals, and the single most important forward-looking statement.
- `qa_defensiveness` — integer 0–10: how much more evasive management was in Q&A vs prepared remarks. 0 = equally confident throughout; 10 = stark contrast.

`max_tokens` is 512 to fit all five fields. **Idempotency** (the property that re-running a step is safe — it never creates duplicates): before calling Claude it checks `scores` and skips records already scored with `trade_brief` present; records predating the extended prompt (no `trade_brief`) or flagged `_mock` are re-scored in place. **Error handling:** malformed responses → log and skip permanently; credit depletion → commit and skip cleanly (re-score later via `/rebackfill-all`); transient/network errors → don't commit, retry on restart. **Output:** writes all five fields to `scores` (unique index `filing_id` , stores `model_used` + `scored_at`) *and* publishes the enriched message to `scored-transcripts` .

Correlation Service — three concurrent threads in one container

- **Main thread** — consumes `scored-transcripts` , fetches yfinance, computes 1d/3d/7d returns, builds `price_series` , resolves sector via the 3-step cascade, computes trend, passes through `guidance_flag` / `trade_brief` / `qa_defensiveness` from the Kafka message, writes the full record to `price_reactions` (falls back to `(ticker, call_date)` upsert key when `filing_id` is absent), and fires notifications. Skips records already in `price_reactions` with `trade_brief` present; re-correlates those missing it or flagged `_mock` .
- **Cache thread** — consumes `raw-prices` , upserts rows into `raw_prices` for BFF chart queries.

- **Backfill daemon** — every 4 hours, finds `price_reactions` with any null returns or null `price_series` and re-fetches yfinance to fill them in (records `backfilled_at`); also backfills missing `sector` values.

Trend signal — compares the current call's score to the same ticker's most recent previous call: "up" ($\geq +5$), "down" (≤ -5), or "neutral". **price_series** — a per-day percentage-change array on each document, aggregated by the BFF sector endpoints into mean $\pm$ std-dev drift paths.

Node.js BFF

`/inspect` **endpoint** — accepts a POST with call metadata, opens an SSE connection, and proxies Claude Sonnet's streaming response back to the browser as JSON text chunks. Rate-limited to 10 requests/hour per user. Uses `INSPECT_MODEL` (default `claude-sonnet-4-6`).

`/companies/:ticker/ingest` — auth-required POST that forwards to the ingestor's `/trigger/<TICKER>` API (`INGESTOR_URL` , default `http://ingestor:8001`). Lets the frontend trigger an immediate scan for any company a user just bookmarked.

`/companies/rebackfill` — auth-required POST that forwards to `/rebackfill-all` . The primary way to backfill `guidance_flag` , `trade_brief` , and `qa_defensiveness` onto older records after an Anthropic credit top-up.

Sector analytics — `GET /sectors` returns per-sector averages; `GET /sectors/:sector` returns a drift path (mean $\pm$ std per day across all company `price_series` in that sector) plus per-company rankings.

EDGAR-preferred deduplication — all aggregations (feed, portfolio, leaderboard, search) dedup by (`ticker` , `call_date`) , preferring EDGAR over FMP so real company names beat ticker-as-name FMP entries. **price_reactions as the read model** — almost every BFF query reads from this single denormalised collection (everything is pre-joined into one ready-made document, so reads need no lookups across other collections); no joins at request time.

5 MongoDB data model

`price_reactions` is the central read model — one denormalised document per filing containing the score, key phrases, guidance flag, trade brief, Q&A defensiveness, returns, trend, `price_series` , and sector. The BFF reads almost exclusively from it, with no joins. Six collections are created by `mongo/init/01_init.js` on first startup; the two internal collections are created on first write.

price_reactions

Primary BFF read source. One doc per filing: confidence_score, key_phrases, guidance_flag, trade_brief, qa_defensiveness, 1d/3d/7d returns, trend, price_series (per-day pct array), call_date_close, sector, company_name, model_used, scored_at, correlated_at, backfilled_at. Written by correlation-service. Unique index: filing_id.

companies

Ticker → name → sector → exchange → CIK. Seeded from EDGAR's ~10k US-listed universe at ingestor startup. This defines what is searchable — not yfinance. International stocks, ETFs, and crypto are outside this universe.

scores

Raw scoring output per filing_id (unique). Written by scoring-service. Stores confidence_score, key_phrases, guidance_flag, trade_brief, qa_defensiveness, model_used, scored_at. Used for the idempotency check before calling Claude — not queried by the BFF directly.

raw_prices

OHLCV cache (ticker + date unique index). Written by correlation-service's cache thread consuming raw-prices. Read only by the BFF `/prices/:ticker` chart endpoint.

users

Email, bcrypt password, watchlist array, notifications_enabled, notifications_email. Read by ingestor (ticker expansion), correlation-service (who to email), and BFF auth/portfolio routes.

transcripts

Optional raw transcript storage. The main pipeline passes text through Kafka — this collection is not in the critical path.

_ingested_filings · internal

Created on first write. Unique index on filing_id. The ingestor writes here before publishing to Kafka — prevents re-publishing across scheduled runs and on-demand triggers. Also used for cross-source (EDGAR/FMP) dedup by call_date.

_sent_notifications · internal

Created on first write. Unique compound index on (user_id, ticker, call_date). Prevents duplicate email sends on correlation-service restart or backfill re-runs.

6 BFF API endpoints

All served by the Node.js Express BFF on port 3001. **PUBLIC** needs no auth · **AUTH** needs a JWT Bearer token · **OPTIONAL** returns richer data when authenticated.

Reads — feed & companies

GET PUBLIC

/feed

Recent correlated calls within 12 months, newest-first. Dedup by (ticker, call_date) — EDGAR preferred.

`?since=` filter. Includes `pending: true` for calls <7 days old still with null returns. Returns per-ticker historical stats (hist_avg_7d, hist_win_rate, hist_call_count). Max 50.

GET PUBLIC

/search?q=

Two-phase: (1) queries `companies` (ticker prefix / name substring) across the full ~10k EDGAR universe, (2) fetches the latest price_reaction per match.

`has_data: true` means a scored + correlated call exists. Data-bearing results first; exact ticker floated to top.

GET PUBLIC

/companies/:ticker/history

All calls for a ticker, newest-first, deduplicated by call_date (EDGAR preferred).

GET PUBLIC

/companies/:ticker/latest

Most recent call for a ticker.

GET PUBLIC

/companies/:ticker

Company metadata: name, sector, exchange, CIK.

GET PUBLIC

/companies/:ticker/accuracy

Track-record analysis in three score buckets (High ≥ 70 · Mid 45–70 · Low <45). Per bucket: call count, avg 1d/3d/7d returns, win rate, avg win/loss, and expected value $EV(7d) = (Win\% \times Avg\ Win) + (Loss\% \times Avg\ Loss)$.

GET PUBLIC

/prices/:ticker?days=N

OHLCV chart data from the `raw_prices` cache. Default 90 days, max 365. Sorted oldest → newest.

POST AUTH

/companies/:ticker/ingest

Triggers an immediate on-demand scan by forwarding to the ingestor's `/trigger/<TICKER>`. Runs EDGAR 90-day + full FMP archive + Alpha Vantage 8 quarters + async backfill. Used when a user first bookmarks a company.

POST AUTH

/companies/rebackfill

Forwards to `/rebackfill-all`. Clears processed flags for every tracked ticker and re-ingests from scratch so existing records re-score with the latest prompt. Primary way to backfill the newer scoring fields after an Anthropic credit top-up.

Reads — aggregates

GET PUBLIC

/leaderboard

All tickers with ≥ 1 fully-windowed call (return_7d not null), dedup by (ticker, call_date). Per ticker: call count, avg confidence, avg 1d/3d/7d returns. Sorted by avg return_7d DESC.

GET PUBLIC

/pulse

Sector-level aggregates: avg confidence and avg return_7d per sector, dedup by (ticker, call_date). Sorted by avg confidence DESC.

GET PUBLIC

/sectors

Summary table of all sectors: avg 1d/3d/7d returns, win rate, call count, company count. One row per sector, sorted by avg 7d return DESC.

GET PUBLIC

/sectors/:sector

Sector deep-dive: drift path (mean $\pm$ std per day from all companies' price_series — $\text{std} = \sqrt{E[X^2] - E[X]^2}$) plus per-company rankings by avg 7d return.

GET OPTIONAL

/calendar?from=&to=

FMP earnings calendar (default next 30 days). If authenticated, marks watchlist tickers
source='portfolio' and server tickers
source='system' (portfolio $\rightarrow$ system $\rightarrow$ rest ordering). Includes EPS/revenue estimates; for tracked tickers, injects avg_score, avg_return_7d, win_rate_7d.

Auth & portfolio

POST PUBLIC

/auth/register

Create account (email + password ≥ 8 chars). Returns 7-day JWT. Rate-limited (20 req/15 min).

POST PUBLIC

/auth/login

Authenticate. Returns 7-day JWT.

GET AUTH

/auth/me

Current user's email, notifications_enabled, notifications_email.

PATCH AUTH

/auth/preferences

Update notifications_enabled (bool) and notifications_email (string).

GET AUTH

/portfolio

Most recent call for each ticker in the user's watchlist, deduplicated and EDGAR-preferred.

POST AUTH

/portfolio/:ticker

Add ticker to watchlist. Uses MongoDB \$addToSet — idempotent.

DELETE AUTH

/portfolio/:ticker

Remove ticker from watchlist.

GET AUTH

/suggestions

Up to 6 companies not in the watchlist, prioritised by sector match then highest confidence.

POST AUTH

/inspect

Streams a 5-section Claude Sonnet deep analysis via SSE. Rate-limited to 10 req/hour. Streams JSON chunks; ends with [DONE] .

7 Frontend pages

ROUTE	DESCRIPTION
/	Dashboard feed. Three time tabs — Live (<24h), This Week, Earlier — auto-selecting the most relevant on load. All vs Portfolio toggle (authenticated). Auto-polls every 60s: incremental update on most polls, full refresh every 5th. Pending badge for calls still filling in returns. Score threshold legend (≥ 70 High / 45–69 Moderate / < 45 Low).
/companies/[ticker]	Stat strip: Latest Score, Calls Tracked, Last Report, Last 7d Return, Predictability (Pearson r), Score Trend, vs Sector 7d. Trade Brief card with guidance badge. Score + return history chart (≥ 2 calls). Post-Earnings Drift chart (current vs historical avg ± 1 std band, sector reference line). CEO Predictability Scatter (confidence vs return, OLS line, Pearson r , R^2 , predicted return). Track Record table (score buckets with win %, avg win/loss, EV). Call cards with GuidanceBadge, QABadge, sparkline, key phrases, 1d/3d/7d returns. Clicking a card opens InspectModal. Bookmarking triggers on-demand ingest. Hint tooltips explain Pearson r , R^2 , EV, score trend, sector delta.
/portfolio	Auth-protected. Watchlist cards + "just reported" section + sector-matched suggestion cards + notification preferences editor (toggle + custom email).
/leaderboard	Ranked table of all tickers by avg 7-day post-call return. Score threshold legend in header. Columns: rank badge, ticker, call count, avg confidence, avg 1d/3d/7d returns, win rate %. Clicking a row opens a drift chart overlay.
/sectors	Sector Pulse. Cards showing avg returns, win rate, call count. Clicking a card expands an inline detail panel on the same page — drift chart (mean ± 1 std band, D0→D7) and per-company rankings. No separate route.

ROUTE	DESCRIPTION
<code>/calendar</code>	FMP earnings calendar (next 30 days). Highlights portfolio and server-tracked tickers. For tracked tickers with history, shows Avg Score, Avg 7d Return, Win % so you can assess track record before the next report. Shows EPS/revenue estimates. Portfolio → system → rest ordering.
<code>/login</code>	Email + password form. Stores JWT in localStorage on success.
<code>/register</code>	Account creation form. Error on duplicate email.

8 Key design decisions

The "why" behind the structure — the non-obvious choices that make the pipeline robust.

Three-pass ingest, not one source

EDGAR is authoritative but only covers 8-K exhibit text. FMP provides structured archives going back years and always runs as Pass 2, even when EDGAR succeeds, because they cover different historical ranges. Alpha Vantage runs only on-demand (25 req/day) for single-ticker triggers. Cross-source dedup happens at two levels: `_ingested_filings` (`filing_id`) and a `call_date` check against `price_reactions`.

Two Kafka topics from the ingestor, not one

The ingestor publishes `raw-transcripts` and `raw-prices` simultaneously. This decouples price caching (correlation-service background thread) from transcript processing (scoring pipeline) — each consumer can fail and retry independently without blocking the other. A separate `price_refresh` job re-publishes OHLCV every 2h for any filing with null returns, so windows fill in automatically.

Scoring service publishes back to Kafka, not straight to Mongo

The scoring service outputs to `scored-transcripts`, not directly to the database. This keeps the pipeline asynchronous: the correlation service can lag, replay messages, or restart without losing data, and the two stages scale independently.

Two Claude models, two purposes

Batch scoring uses `SCORING_MODEL` — the recommendation in `.env.example` is Haiku for cost efficiency (prompt caching means the system prompt is billed once per cache window). The docker-compose default when no `.env` is present is Sonnet. Sonnet via `INSPECT_MODEL` is used only when a user opens the InspectModal — interactive, on-demand, rate-limited to 10 req/hour.

Five scoring fields, not one

The original prompt returned only `confidence_score` and `key_phrases`. It was extended to return three more in the same Claude call at no extra latency: `guidance_flag`, `trade_brief`, and `qa_defensiveness`. All five flow through the same Kafka message onto every `price_reactions` document, so the frontend renders a guidance badge, trade brief, and Q&A caution badge with no extra API calls. Records predating the extension are re-scored automatically when they next appear in the queue; a full re-score is triggered via `POST /companies/rebackfill`.

On-demand ingest via HTTP trigger

The ingestor exposes a Flask API on port 8001. The BFF's `POST /companies/:ticker/ingest` forwards to it when a user bookmarks a company with no data yet — giving near-immediate feedback instead of waiting up to `SCHEDULE_INTERVAL_HOURS`. The trigger runs Pass 1+2+3 synchronously, then kicks off a full historical backfill in the background.

How an unmonitored company gets data

Searching does *not* trigger ingestion — it only reads what exists (`has_data: false` for un-ingested companies). **Adding to a watchlist is the primary trigger:** the BFF calls `/companies/:ticker/ingest` for immediate processing, and the ingestor also merges every user's watchlist into its scheduled scan set. Once added: immediate trigger scan (EDGAR 90d + full FMP + Alpha Vantage 8q) → async full historical backfill → the RSS poller covers all future filings in real time. The result is the same full card as any monitored company, typically within minutes.

RSS poller as the real-time layer

The original design relied purely on a per-ticker APScheduler scan. The RSS 8-K Atom poller was added for near-real-time coverage of *any* public company — not just those in `TICKERS`. The per-ticker scan remains as a reliability backstop for anything the feed misses.

price_reactions as the denormalised read model

Rather than joining scores + transcripts + prices at query time, the correlation-service writes a single document per call into `price_reactions` with everything the frontend needs. The BFF reads almost exclusively from this one collection — no cross-collection joins on any request path.

Sector drift path via price_series

Each `price_reactions` doc stores a `price_series` array (per-day % change from the call-date baseline). The BFF `/sectors/:sector` endpoint aggregates all companies' arrays within a sector into a mean drift path and confidence band ($\text{std} = \sqrt{E[X^2] - E[X]^2}$) without storing any sector-level summary documents — so sector charts are always live and auto-include newly correlated calls.

Idempotency at every stage

`_ingested_filings` prevents re-publishing to Kafka (covers scheduled runs and HTTP triggers). The scoring-service checks `scores` before calling Claude — skips if all five fields present, re-scores if `trade_brief` is absent or `_mock`. The correlation-service upserts keyed on `filing_id`. `_sent_notifications` has a unique compound index on (user_id, ticker, call_date). Every service can restart, crash, or replay messages without producing duplicate data.

9 Local infrastructure — Docker Compose

The full stack runs locally as ten containers. This is what the AWS deployment (next section) mirrors.

```
# infrastructure
zookeeper      # Kafka dependency (Confluent 7.5)
kafka          # Broker - 3 topics, auto-create disabled
```

```

kafka-setup      # One-shot: creates the 3 topics, then exits
kafka-ui         # Provectus web UI – port 8080
mongo            # mongo:7 – init script creates 6 collections + indexes

# application services
ingestor         # Python – EDGAR + FMP + Alpha Vantage + RSS + price_refresh + Flask (8001)
scoring-service  # Python – Claude scoring – 5 fields
correlation-service # Python – returns + price_series + sector + raw_prices cache + 4h backfill
bff             # Node.js Express – port 3001
frontend        # Next.js 14 – port 3000

volumes:
  mongo_data      # MongoDB persistence across restarts
  kafka_data      # Kafka broker logs
  zookeeper_data  # Zookeeper data persistence
  zookeeper_log   # Zookeeper transaction logs

```

10 AWS deployment

infra/terraform/*.tf

On AWS, the same containers run in the cloud, reachable from the internet and described entirely as code with Terraform. `terraform apply` builds everything; `terraform destroy` tears it down. CI/CD (GitHub Actions, manual `workflow_dispatch`) builds the images, pushes them to ECR, and forces a new ECS deployment. Almost every AWS piece maps to something from the local stack above.

The big picture — local vs AWS

LOCAL (DOCKER COMPOSE)	AWS	ROLE
5 service containers	ECS Fargate	Runs the containers — no servers to manage
Docker images on disk	ECR	Private registry storing the built images
<code>confluentinc/cp-kafka</code>	MSK	Managed Kafka — the message bus
<code>mongo:7</code> container	MongoDB Atlas (not AWS)	The database
<code>localhost</code> ports	2 × ALB	Public URLs into the frontend and the BFF

LOCAL (DOCKER COMPOSE)	AWS	ROLE
<code>.env</code> file	Secrets Manager	API keys & passwords, injected at runtime
files on disk	S3	Archives raw transcripts
<code>docker logs</code>	CloudWatch	Logs, dashboard, alarm
your laptop's network	VPC	The private network everything lives in
—	IAM	Permissions: who is allowed to do what

VPC — the private network · `vpc.tf`

An isolated network inside AWS, range `10.0.0.0/16`, spread across **two Availability Zones** (two physical data centres) with two kinds of subnet:

- **PUBLIC** `10.0.0.x / 10.0.1.x` — internet-facing. **Only the two load balancers** live here.
- **PRIVATE** `10.0.10.x / 10.0.11.x` — **all five containers and the Kafka brokers** live here, unreachable from the internet directly.

Two supporting pieces: the **Internet Gateway** (the door between public subnets and the internet, so the ALBs can receive traffic) and the **NAT Gateway** (lets private containers make *outbound* calls to Anthropic, yfinance, EDGAR, and Atlas without being reachable inbound). The NAT's fixed IP is exported as the `nat_gateway_ip` output — you add it to **MongoDB Atlas** → **Network Access** so containers can reach the database.

ECS Fargate — runs the containers · `ecs.tf`

ECS orchestrates containers; **Fargate** is the serverless mode where AWS supplies the compute. A **task definition** is the cloud equivalent of one `docker-compose.yml` service block; a **service** keeps a task running (`desired_count = 1` restarts it if it dies).

SERVICE	CPU / RAM	PUBLIC?	TALKS TO
ingestor	512 / 1024	No	EDGAR, FMP → S3 + MSK + Mongo
scoring-service	512 / 1024	No	MSK, Anthropic API, Mongo

SERVICE	CPU / RAM	PUBLIC?	TALKS TO
correlation-service	256 / 512	No	MSK, Mongo, Resend
bff	256 / 512	Yes (BFF ALB)	Mongo
frontend	512 / 1024	Yes (Frontend ALB)	the BFF

A one-shot `kafka-setup` task creates the three topics after MSK is ready, then exits.

`ignore_changes = [task_definition, desired_count]` tells Terraform to leave running versions alone — CI/CD deploys new images, not Terraform.

MSK — managed Kafka · `msk.tf`

The cloud version of the local Kafka container: **2 brokers** (`kafka.t3.small` , one per AZ) in the private subnets, PLAINTEXT/unauthenticated but locked down by its security group so **only the ECS containers** can reach it.

⚠️ **This is the most expensive piece (~\$93/month) and it bills even when idle** — the main reason the stack is designed to be `terraform destroy` -ed when not in use.

ALB — the public front doors · `alb.tf`

Note: there are **two separate Application Load Balancers** (`aws_lb.frontend` and `aws_lb.bff`), each with its own target group and listener — not one ALB with two target groups.

Each ALB gives part of the app a public URL and health-checks its target: the **Frontend ALB** → frontend container (port 3000, health check `/`); the **BFF ALB** → BFF container (port 3001, health check `/health`). They listen on port 80 and auto-add an HTTPS:443 listener (with HTTP→HTTPS redirect) if an ACM certificate ARN is supplied.

Why two ALBs, not one?

Next.js bakes its API base URL (`NEXT_PUBLIC_API_URL`) into the frontend **at build time**, so the build needs the BFF's public address *before* the frontend image is built. The BFF ALB is created first, its DNS name is exported as the `bff_url` output, and CI/CD feeds it in as a build argument for the frontend image. Two separate front doors is what makes that possible.

Supporting services

S3 · s3.tf

Object storage for raw transcript archives (best-effort; no-op locally). Versioned, encrypted (AES256), fully private. Lifecycle: → Glacier after 90 days, deleted after 1 year.

ECR · ecr.tf

Private registry, one repo per service. Scans images on push; auto-deletes untagged images after 1 day.

Secrets Manager · secrets.tf

Holds five secrets (mongo-uri, anthropic-api-key, jwt-secret, fmp-api-key, resend-api-key); ECS injects each into the right container at startup. Never baked into images or git.

IAM · iam.tf

Two least-privilege roles: the **execution role** (start the container — pull image, read secrets, write logs) and the **task role** (runtime — ingestor writes S3, services push metrics).

CloudWatch · cloudwatch.tf

Per-service log groups (14-day retention), an `esp-overview` dashboard (one panel per service), and an alarm if the scoring service has zero running tasks for 2 minutes.

Security Groups · security_groups.tf

Per-resource firewalls: ALBs accept from anyone; ECS accepts only from the ALBs; MSK accepts only from ECS. Traffic can never skip a layer.

How traffic flows on AWS

Internet → ALB **PUBLIC** → ECS containers **PRIVATE** → MSK / Mongo **PRIVATE**

User loading the site:

Browser → Frontend ALB → Frontend container
→ (browser then calls) BFF ALB → BFF container → MongoDB Atlas → back to browser

Background processing (all outbound via NAT Gateway):

Ingestor → archives to S3 + publishes to MSK
Scoring → consumes MSK → Anthropic API → writes Mongo + publishes to MSK
Correlation → consumes MSK → price reaction → writes Mongo → emails via Resend

What it costs · us-east-1, while running

RESOURCE	APPROX COST	NOTES
MSK — 2 × kafka.t3.small	~\$93 / mo	Bills 24/7, idle or not

RESOURCE	APPROX COST	NOTES
ECS Fargate — 5 tasks	~\$36 / mo	Scales with declared CPU/RAM
NAT Gateway	~\$33 / mo + data	Bills 24/7 , plus \$0.045/GB
2 × ALB	~\$12 / mo + LCU	One per front door

≈ **\$150–200 / month while running**. MSK and the NAT Gateway are the always-on charges — which is why the stack is built to be `terraform destroy`-ed when idle and rebuilt with `terraform apply` when needed (re-whitelist the new NAT IP in Atlas afterwards).

What this means for you

Leaving the stack up does nothing useful while no one is using it, but still costs ~\$5–7 a day — mostly MSK and the NAT Gateway, which bill by the hour whether or not traffic flows. The practical habit: `terraform apply` to bring it up for a demo or test session, then `terraform destroy` when you're done. Your live application data is safe — it lives in **MongoDB Atlas**, which is external to this Terraform and is never torn down. (The S3 transcript bucket is part of the managed infrastructure and would be removed on destroy, but it only holds best-effort raw-transcript backups — not the data the app actually serves.) The only manual step after each rebuild is copying the new `nat_gateway_ip` into Atlas → Network Access so the containers can reach the database again.

11 Environment variables

The complete configuration surface. On AWS, the secret values come from Secrets Manager; the rest are set as ECS task environment variables.

```
# MongoDB
MONGO_ROOT_USER=admin
MONGO_ROOT_PASSWORD=changeme
MONGO_URI=mongodb://admin:changeme@mongo:27017/earnings_sentiment?authSource=admin

# Kafka
KAFKA_BOOTSTRAP_SERVERS=kafka:29092

# SEC EDGAR (required by fair-use policy)
EDGAR_USER_AGENT="YourName contact@email.com"
```

```
# Ingestor
TICKERS=AAPL,MSFT,GOOGL,AMZN,META,NVDA,TSLA,JPM,JNJ,XOM
LOOKBACK_DAYS=30          # docker-compose default 730; bare code default 15000
SCHEDULE_INTERVAL_HOURS=2
RSS_POLL_INTERVAL_MINUTES=10

# FMP (enables transcript Pass 2 + /calendar endpoint)
FMP_API_KEY=

# Alpha Vantage (enables transcript Pass 3 – on-demand only, 25 req/day free tier)
ALPHAVANTAGE_API_KEY=

# Claude / Anthropic
ANTHROPIC_API_KEY=sk-ant-...
SCORING_MODEL=claude-haiku-4-5-20251001  # recommended; docker-compose fallback claude-sonnet-
INSPECT_MODEL=claude-sonnet-4-6          # used by BFF /inspect
MAX_TRANSCRIPT_CHARS=150000              # scoring-service hard cap on text sent to Claude

# Correlation
PRICE_FETCH_DAYS=12

# BFF
JWT_SECRET=use_a_long_random_string_here
PORT=3001
CORS_ORIGIN=http://localhost:3000
INGESTOR_URL=http://ingestor:8001        # BFF uses this to call /trigger/

# Email notifications (set NOTIFICATION_PROVIDER=resend to enable)
NOTIFICATION_PROVIDER=none
RESEND_API_KEY=
APP_URL=http://localhost:3000
NOTIFY_FROM_EMAIL=onboarding@resend.dev

# Frontend (Next.js build-time)
NEXT_PUBLIC_API_URL=http://localhost:3001
```